

Group 10 Analysis Report

Azure Sands Resort & Spa Information System

Isable Kim, Kalman Dong, William Jiang, Shirley Yu

New York University

CSCI-UA-60: Database Design and Implementation

Prof. Mark Ramos

Apr 28, 2026

I. Introduction

Our project models a database-driven information system for a resort hotel that supports lodging, event hosting, facility management, and billing operations. The goal of the system is to organize complex and interconnected activities, such as reservations, guest stays, room assignments, event bookings, service usage, and payments, into a structured relational database that supports coordination and operational decision-making.

Rather than treating the hotel simply as a booking system, we approached it as an integrated organizational system involving multiple resources, time-based usage, and different stakeholder roles, including guests, organizations, billed parties, and hotel management.

The final design includes 23 normalized tables, covers:

- physical hotel hierarchy
- room structure and adjacency
- reservations and stays
- event and facility usage
- billing and payments

II. Key Design and Decisions

a. Separation of Reservations and Stays

One major design decision was distinguishing between reservations and stays.

We treated a reservation as an intended future booking, while a stay represents actual occupancy after check-in. This distinction allows the system to capture changes between planned and actual room usage, supports walk-ins or modifications, and reflects real hotel operations more accurately.

Related supporting tables include:

- reservation
- reservation_guest
- reservation_room_request
- stay
- stay_room

This was one of the core structural decisions of the model.

b. Modeling the Physical Hotel Hierarchy

We modeled the hotel's physical structure as:

- hotel_complex
- building
- wing
- floor
- room

This hierarchy allows rooms to inherit spatial and functional characteristics (e.g. near pool, near parking, accessibility), supports occupancy analysis by location, and enables facility/event scheduling at the room level.

We chose to preserve this hierarchy rather than flattening room data into a single table because it better supports resource allocation queries.

c. Separate Billing Pipeline

Another major decision was separating financial activity into:

- billed_party
- charge
- payment
- deposit

We intentionally separated:

- charges = amounts billed
- payments = amounts received

rather than combining them into one transaction table. This separation supports balance calculations, partial payments, deposit tracking, and more realistic accounts receivable management, allowing the system to represent financial activity beyond simple one-time transactions.

We also clarified the distinction among:

- service_item = service definition
- facility_booking = use of facilities for events
- charge = financial record

This revision was made partly in response to feedback about reducing ambiguity in billing-related tables.

d. Event and Facility Booking Design

We modeled events separately from reservations and linked them through facility bookings:

- event
- facility_booking

This supports business use cases where organizations use hotel resources for conferences, retreats, or workshops without those activities necessarily functioning as guest stays.

It also enabled event utilization and revenue analysis queries.

III. Normalization and Data Modeling Assumptions

The schema was designed in Third Normal Form (3NF).

Examples:

- Guest personal data is stored in person, avoiding duplication.
- Organizations are separated from individuals.
- Bed configurations are normalized through bed_type and room_bed.
- Many-to-many relationships are resolved using junction tables such as: reservation_guest, room_bed, room_adjacency.

This reduces redundancy and improves consistency.

IV. Assumptions & Ambiguity

Several parts of the case description were ambiguous, so we made and documented the following assumptions.

a. Pricing Rules

Dynamic pricing and assignment prioritization were not modeled directly in the database.

Instead, room rates are stored as base rates within the schema, while more advanced pricing logic, such as seasonal pricing adjustments, demand-based pricing, or room assignment prioritization, is assumed to be handled through application logic rather than enforced at the database level. This was a deliberate design decision to keep the schema focused on core data

relationships while acknowledging that more sophisticated pricing and allocation logic could be incorporated as a future extension.

b. Charge Relationships

The case description could support highly complex billing relationships, particularly because charges may arise from lodging, event facility usage, or additional services. To keep the model implementable and reduce ambiguity, we simplified the design so that each charge is associated primarily with one billing context, either a stay charge, a facility booking charge, or a service charge. This provides each charge record with a clearer meaning while still preserving enough flexibility to support multiple revenue sources within the system. This simplification also helped align the billing model with project scope while maintaining realistic financial tracking.

c. Core Scope vs Extensions

Following milestone feedback, we prioritized a core implementation:

Fully implemented:

- physical hierarchy
- reservations and stays
- events and facility bookings
- billing

Reduced or omitted as extensions:

- detailed access control
- activity logs
- more advanced service ordering workflow

This kept scope manageable while preserving system integrity.

d. Time-Based Event Scheduling Simplification

A separate meeting_time_slot table was considered earlier in the design process but was removed in the final model.

Instead, event timing is represented through:

- bookingStartDateTime
- bookingEndDateTime

within facility_booking.

This simplified implementation while still supporting scheduling and event utilization analysis.

V. Queries and Information Interpretation

The queries focus on:

- occupancy monitoring
- reservation demand
- event utilization
- charges and payments
- outstanding balances
- revenue composition
- service category performance

Rather than simply retrieving records, the queries aggregate and interpret data to support management decisions.

Examples:

- occupancy by wing supports staffing and resource allocation
- outstanding balances support financial tracking
- service revenue identifies profitable offerings
- event usage supports facility planning

These queries directly drive the web interface dashboards.

VI. Web Interface

The web interface was designed as two dashboard pages.

(i) Operations Dashboard:

- occupied rooms
- occupancy by wing
- guest assignments
- event activity
- reservation demand

Purpose: support operational awareness.

(ii) Revenue and Billing Dashboard:

- charges by billed party
- payments received
- outstanding balances
- revenue by category
- service revenue mix

Purpose: support financial monitoring and planning.

VII. Tradeoffs and Limitations

Several tradeoffs were made in order to balance realism with project scope and implementation feasibility. Some real-world features were intentionally simplified, including dynamic pricing, staff scheduling, advanced access control, and a more detailed service workflow. While these features could be important in a production hospitality system, they were treated as possible future extensions so the project could focus on core hotel operations such as reservations, stays, facility usage, and billing.

The sample data included in the database is designed to be realistic and sufficient to support the required analytical queries, though it does not attempt to simulate the full scale or transaction volume of a real resort operation. The focus of the implementation was on correctness of relationships, support for meaningful queries, and interpretability of outputs rather than large-scale operational simulation.

Some business rules are also handled through controlled data input rather than extensive database-level constraint logic. For example, charge categorization, certain preference rules, and some assignment-related logic are maintained through the structure of the data and application assumptions rather than advanced SQL enforcement. Because the project emphasizes coherent system modeling over exhaustive rule enforcement, this was considered an appropriate tradeoff within the course scope. Certain business logic is therefore handled through application logic and controlled sample data rather than more complex database constraints.

VIII. Division of Work

Work was divided collaboratively across the group across several major components of the project, including ERD modeling and relationship design, database schema implementation and

sample data population, query development and testing, web interface dashboard mockups, and analysis report integration. Although responsibilities were distributed among group members, major design decisions were reviewed collectively and refined through group discussion to ensure consistency across the ERD, SQL implementation, queries, and interface design. This collaborative approach helped strengthen the coherence of the final system design.

IX. Conclusion

This project demonstrates how a relational database can model a complex hospitality system involving shared resources, time-based usage, event operations, and financial flows. By combining normalized schema design, analytical queries, and dashboard-style reporting, the system supports coordination, accountability, and decision-making within the organization. The final model balances realism, implementability, and analytical usefulness while addressing ambiguity through documented design choices. Through the integration of reservations, stays, event bookings, service usage, billing, and reporting, the project illustrates how database design can support both operational management and analytical insight within a resort environment.